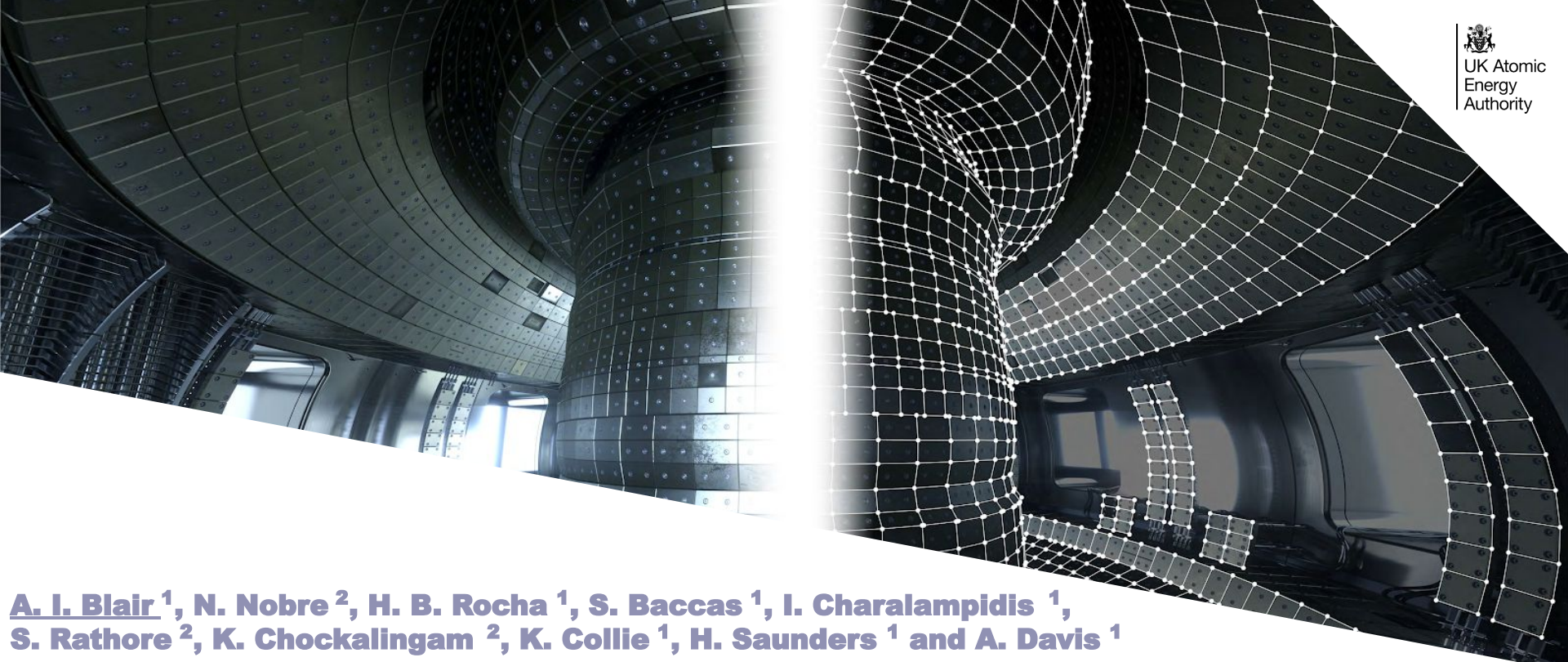




**A. I. Blair¹, N. Nobre², H. B. Rocha¹, S. Baccas¹, I. Charalampidis¹,
S. Rathore², K. Chockalingam², K. Collie¹, H. Saunders¹ and A. Davis¹**

MFEM in MOOSE: A Framework for Coupled Multiphysics

¹ United Kingdom Atomic Energy Authority, Culham Centre for Fusion Energy, Abingdon, UK

² UKRI STFC Hartree Centre, Daresbury, UK

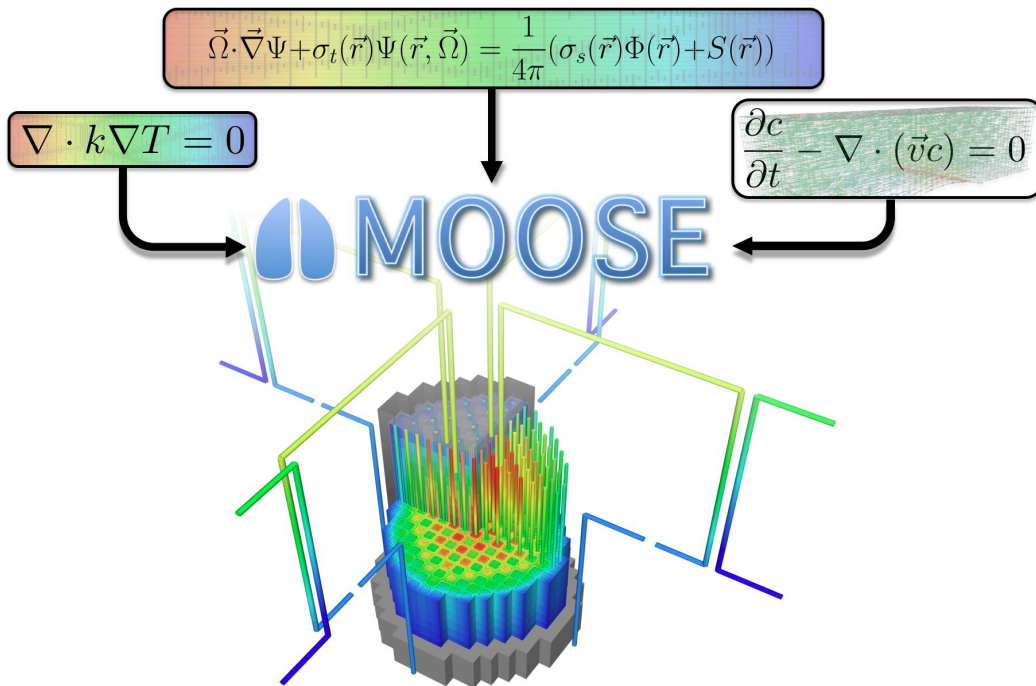
MFEM Community Workshop 2026, Georgia Tech, Georgia, U.S.A

24th September 2026

1. Introduction to MOOSE
2. MFEM in MOOSE
3. Recent Additions
4. Future Work
5. Summary

1. Introduction to MOOSE
2. MFEM in MOOSE
3. Recent Additions
4. Future Work
5. Summary

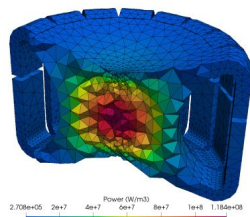
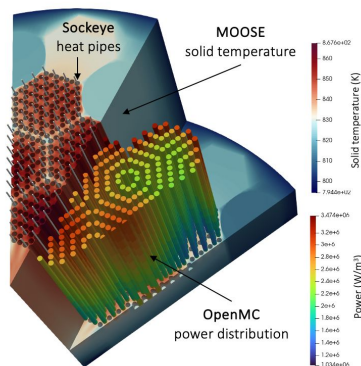
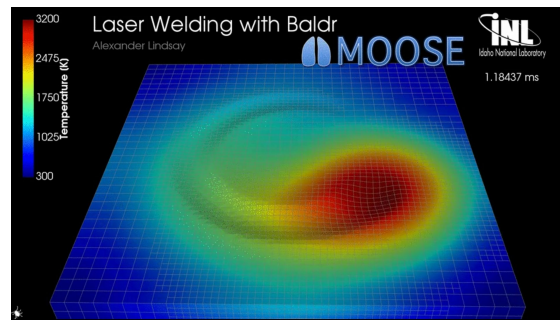
What is MOOSE?



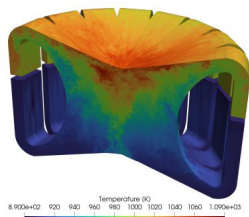
- Open source parallel multiphysics finite element framework from INL
- Built on the libMesh FE library (originally!)
- Active, large user community (>2.3k ★ on GitHub)

Why MOOSE?

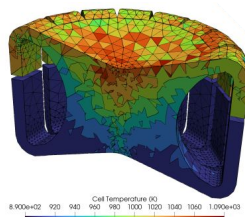
Need for scalability, diverse (coupleable) multiphysics, and simplicity for end user



(a) OpenMC power



(b) NekRS LES fluid temperature



(c) OpenMC cell temperature

Figures from A.J. Novak, H. Brooks, P. Shriwise, A. Hegazy, and A. Davis, "Multiphysics Coupling of OpenMC CAD-Based Transport to MOOSE using Cardinal and Aurora" Proceedings of M&C (2023)

- Modular design with extensive ecosystem of existing MOOSE apps - enabling coupling with external tools like NekRS, OpenMC, OpenFOAM...
- Simple user interface for defining nonlinear multiphysics problems
- Robust software quality assurance procedures - NQA-1 standard
- Wide range of existing validated multiphysics modules

- Chemical reactions
- Combined
- Contact
- Electromagnetics
- Fluid Properties
- Fluid-Structure Interaction
- Functional Expansion Tools
- Geochemistry
- Heat Transfer
- Level Set
- Misc
- Navier-Stokes
- Optimization
- Peridynamics
- Phase Field
- Porous Flow
- Ray Tracing
- Reactor
- Reconstructed Discontinuous Galerkin
- Richards
- Solid Mechanics
- Solid Properties
- Stochastic Tools
- SubChannel
- Thermal Hydraulics
- XFEM

<https://mooseframework.inl.gov/modules/>

1. Introduction to MOOSE
2. MFEM in MOOSE
3. Recent Additions
4. Future Work
5. Summary

Getting Started

MFEM

To enable MOOSE's MFEM capabilities, it is necessary to install all of its dependencies, including MFEM itself and Conduit. You may do so by performing the following steps:

```
./scripts/update_and_rebuild_petsc.sh [options]
```

Copy

You may substitute **[options]** with any PETSc configure flags, which are discussed in the [PETSc install page](#). If you intend to use GPU offloading capabilities, make sure to include **--with-cuda** or **--with-hip**. If desired, you may also specify the GPU architecture with **--with-cuda-arch=[arch]** or **--with-hip-arch=[arch]**. To maximize performance and avoid errors, you should also make sure that your underlying MPI library is GPU-aware.

NOTE

The PETSc GPU architecture specification takes in only the architecture number for CUDA builds, for instance for the **sm_80** architecture you would add the flag **--with-cuda-arch=80**. For HIP builds you should use the entire label, for instance **--with-hip-arch=gfx908**.

Next, we build libMesh, Conduit, and Wasp by running their respective scripts:

```
./scripts/update_and_rebuild_libmesh.sh  
./scripts/update_and_rebuild_conduit.sh  
./scripts/update_and_rebuild_wasp.sh
```

We must then build MFEM itself by running its script:

```
./scripts/update_and_rebuild_mfem.sh [options]
```



TIP

Alternatively, if you already have a working MFEM build in a separate directory, you may set the variable **MFEM_DIR** to its install path instead of running the MFEM install script.

2. Configure the MOOSE build by running

```
./configure --with-mfem
```

You may also wish to include other MOOSE configuration flags as necessary.

3. Finally, build the framework, module, and tests as desired by running

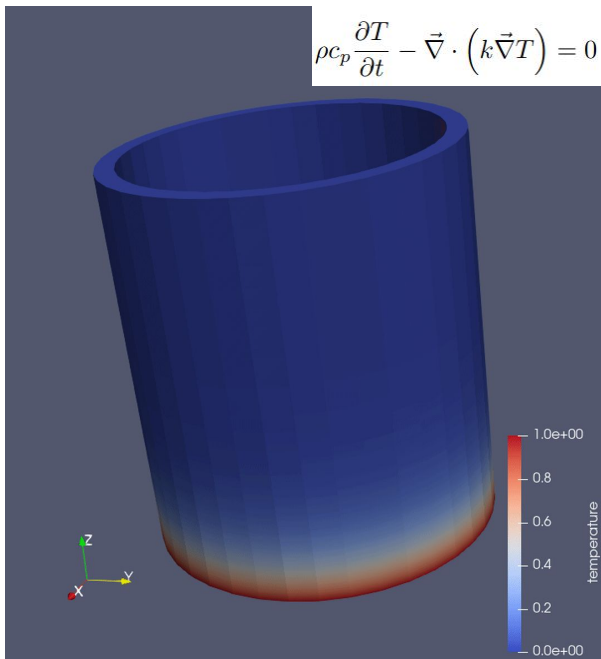
```
cd framework  
make -j $MOOSE_JOBS  
cd ../modules  
make -j $MOOSE_JOBS  
cd ../test  
make -j $MOOSE_JOBS
```

https://mooseframework.inl.gov/getting_started/installation/install_mfem.html

Examples

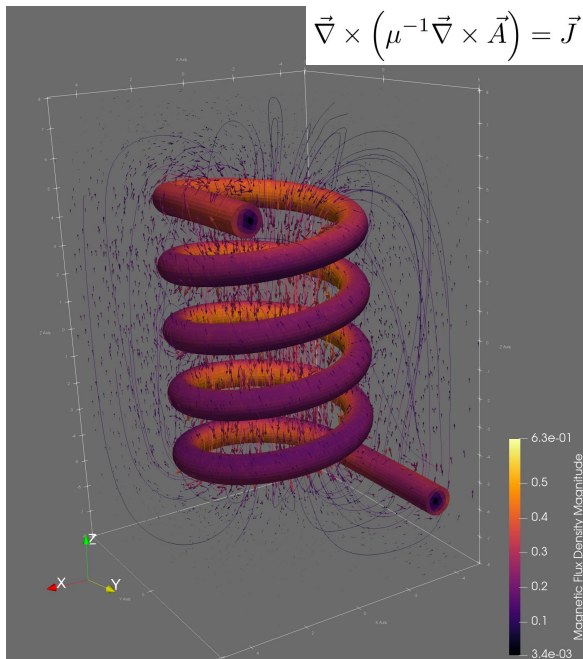
https://mooseframework.inl.gov/syntax/MFEM/examples_index.html

$$\rho c_p \frac{\partial T}{\partial t} - \vec{\nabla} \cdot (k \vec{\nabla} T) = 0$$

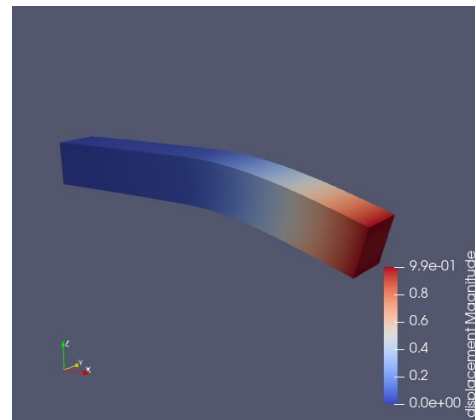


Thermal Physics

$$\vec{\nabla} \times (\mu^{-1} \vec{\nabla} \times \vec{A}) = \vec{J}$$



Electromagnetics



Linear Elasticity

Inputs

```
[Mesh]
  type = MFEMFileMesh
  file = ../mesh/mug.e
[]

[Problem]
  type = MFEMProblem
[]

[FESpaces]
  [H1FESpace]
    type = MFEMScalarFESpace
    fec_type = H1
    fec_order = FIRST
  []
[]

[Variables]
  [concentration]
    type = MFEMVariable
    fespace = H1FESpace
  []
[]

[BCs]
  [bottom]
    type = MFEMScalarDirichletBC
    variable = concentration
    boundary = 'bottom'
    coefficient = 1.0
  []
  [top]
    type = MFEMScalarDirichletBC
    variable = concentration
    boundary = 'top'
  []
[]

[Kernels]
  [diff]
    type = MFEMDiffusionKernel
    variable = concentration
  []
[]

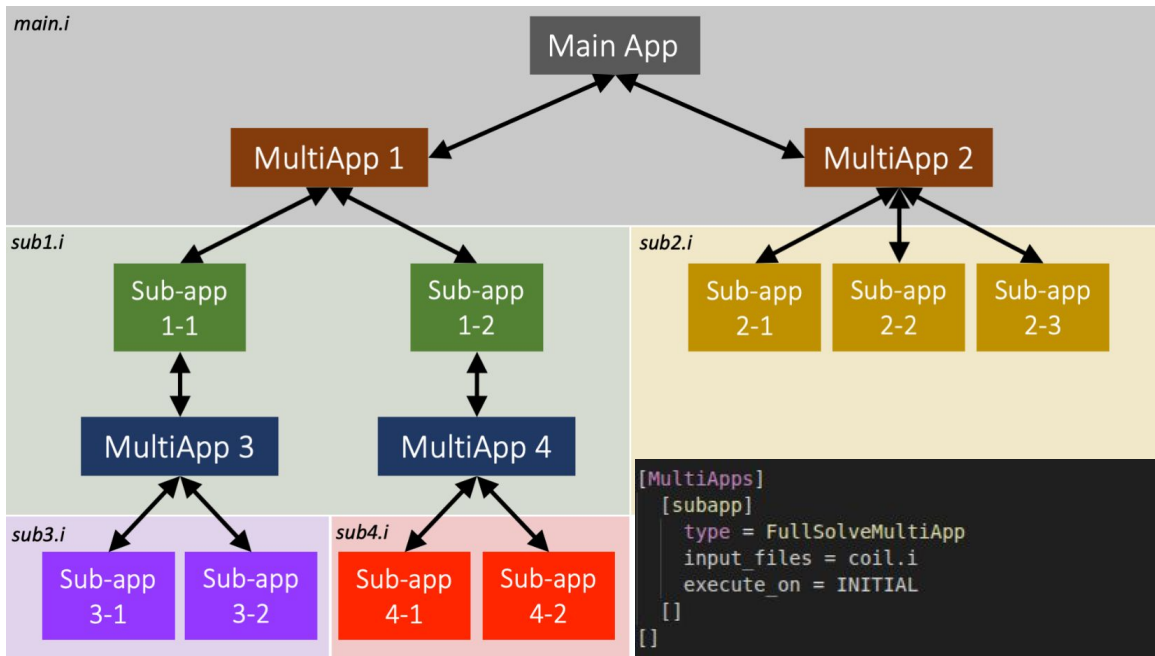
[Solvers]
  [boomeramg]
    type = MFEMHypreBoomerAMG
  []
  [main]
    type = MFEMHypreGMRES
    preconditioner = boomeramg
    l_tol = 1e-16
    l_max_its = 1000
  []
[]

[Executioner]
  type = MFEMSteady
  device = cpu
  assembly_level = legacy
[]

[Outputs]
  [ConduitDataCollection]
    type = MFEMConduitDataCollection
    file_base = OutputData/ConduitDC
  []
[]
```

MOOSE Builder Class	Built MFEM-derived object(s)
MFEMProblem	(all)
MFEMMesh, MFEMSubMesh	mfem::ParMesh, mfem::ParSubMesh
MFEMFESpace	mfem::ParFiniteElementSpace
MFEMVariable	mfem::ParGridFunction
MFEMParsedFunction, MFEMQuadratureFunctionBase,	mfem::Coefficient, mfem::QuadratureFunctionCoefficient
MFEMKernel, MFEMBoundaryCondition	mfem::BilinearFormIntegrator, mfem::MixedBilinearFormIntegrator, mfem::LinearFormIntegrator, mfem::NonlinearFormIntegrator
MFEMWeakForm [†]	Moose::MFEM::EquationSystem (mfem::Operator)
MFEMLinearSolverBase, MFEMNonlinearSolverBase	mfem::Solver
MFEMProblemComposer	Moose::MFEM::ProblemOperator (mfem::Operator)
MFEMDataCollection	mfem::DataCollection

MultiApp Support



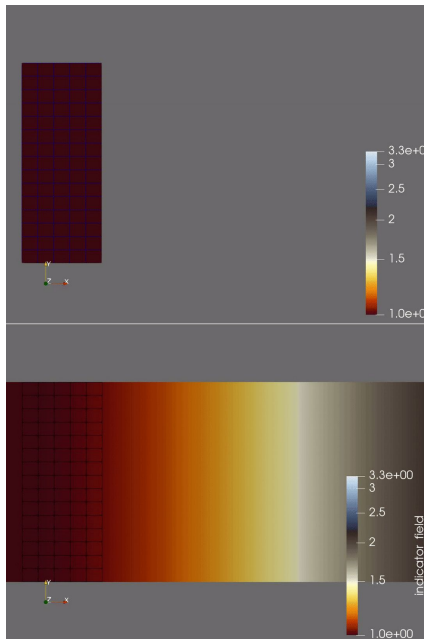
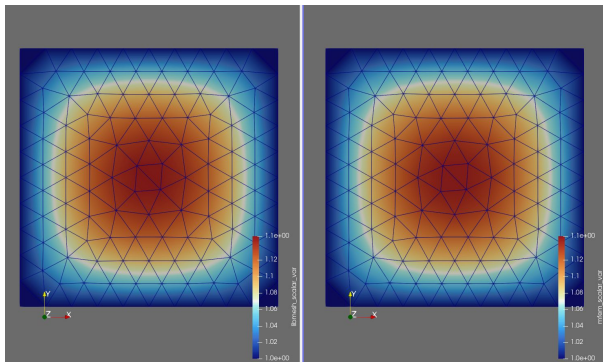
- MOOSE's MultiApp orchestration commonly used to orchestrate execution of (coupled) FE problems
- Helpful for coupling physics on different time and length scales

<https://mooseframework.inl.gov/syntax/MultiApps/>

1. Introduction to MOOSE
2. MFEM in MOOSE
3. Recent Additions
4. Future Work
5. Summary

MFEM $\leftrightarrow$ libMesh Interpolative Transfers

```
[Transfers]
# Nodal to nodal variables
[mfem_to_libmesh_nodal_nodal]
type = MultiAppMFEMToLibMeshShapeEvaluationTransfer
from_multi_app = mfem_sub
to_multi_app = libmesh_sub
source_variables = sent_nodal
variables = received_nodal
[]
[libmesh_to_mfem_nodal_nodal]
type = MultiAppLibMeshToMFEMShapeEvaluationTransfer
from_multi_app = libmesh_sub
to_multi_app = mfem_sub
source_variables = sent_nodal
variables = received_nodal
[]
[]
```

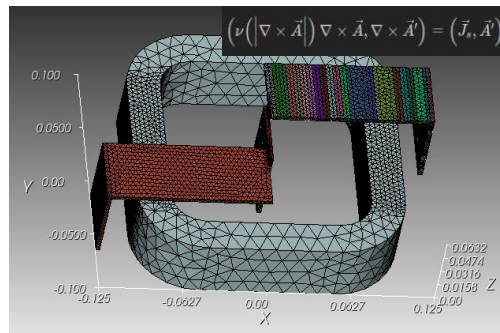


- Interpolative transfers between MFEM and libMesh using GSLib, supporting:
 - First and second order scalar LAGRANGE variables, and lowest order MONOMIAL FEs between libMesh and MFEM
 - Transfers between dissimilar meshes
 - 2D and 3D transfers
- Interpolative MFEM/MFEM transfers using GSLib, supporting:
 - Transfers between scalar and vector FE types
 - Transfers between dissimilar meshes
 - Transfers between submeshes and parent meshes
 - 2D and 3D transfers

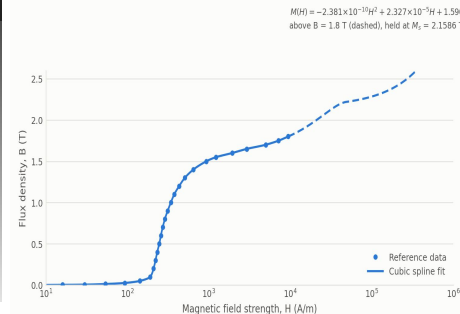
<https://mooseframework.inl.gov/syntax/MultiApps/>

Nonlinear Problems

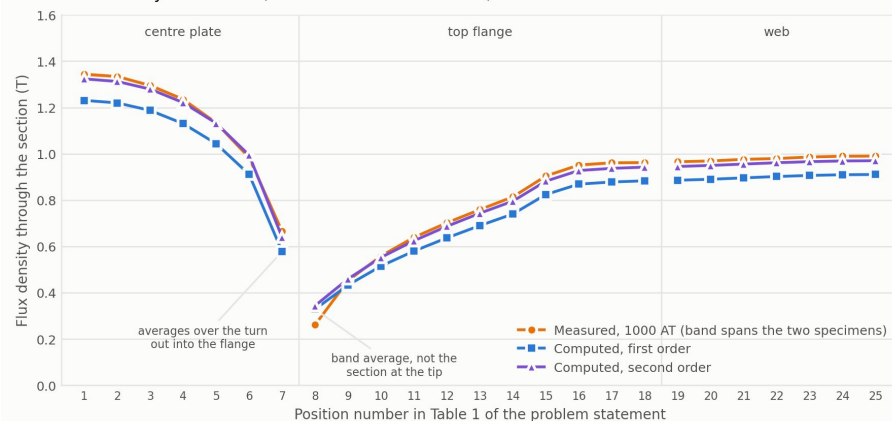
- Support added for `mfem::NonlinearFormIntegrator` contributions in `Moose::MFEM::EquationSystem`
- Nonlinear solvers available from both PETSc and native MFEM
- Nonlinear problems with nonlinear off-diagonal Jacobian contributions not yet supported



B-H curve of the TEAM Problem 13 steel, to saturation



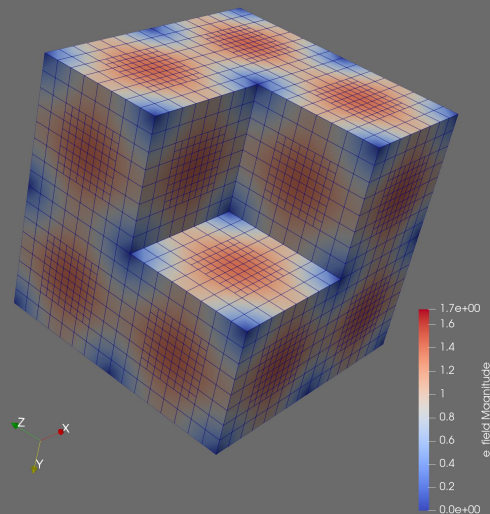
Flux density in the steel, all 25 measured sections, at 1000 AT



TEAM Problem 13 solved in MFEM-MOOSE

<https://github.com/aurora-multiphysics/canary/pull/10>

Adaptive Mesh Refinement



```
[Adaptivity]
[Indicators]
[l2zz]
  type      = MFEMZienkiewiczZhuIndicator
  variable  = e_field
  kernel    = curlcurl
  outputs   = none
[]
[]
[Markers]
[ref]
  type      = MFEMRefinementMarker
  refine    = 0.7 # error
  indicator  = l2zz
  max_h_level = 8
  max_p_level = 1
  refinement_type = h
[]
[]
[]
```

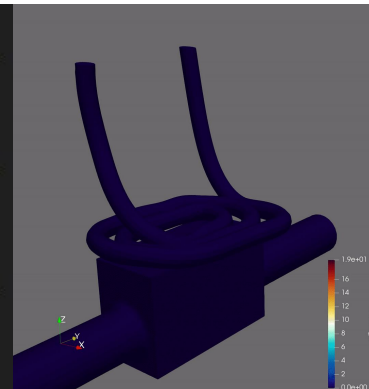
- Initial AMR support has been added to MFEM-MOOSE
- Both h- and p- refinement supported (but not simultaneously)

Mixed Complex & Transient Systems

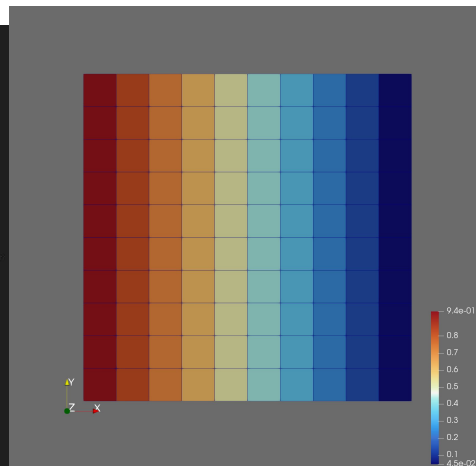
- Mixed complex and transient systems can now be set up from MOOSE inputs
- MixedSesquilinearForm and ParMixedSesquilinearForm added to MFEM to facilitate this

<https://mooseframework.inl.gov/syntax/MultiApps/>

```
[mixed_grad_V]
  type = MFEMMixedSesquilinearFormKernel
  trial_variable = V
  variable = A
  [RealComponent]
    type = MFEMMixedVectorGradientKernel
    coefficient = ${alpha}
  []
  [ImagComponent]
    type = MFEMMixedVectorGradientKernel
    coefficient = ${beta}
  []
[]
```



```
[kernels]
  [dT_dt,T']
    type = MFEMTimeDerivativeMassKernel
    variable = temperature
  []
  [divh,T']
    type = MFEMVectorFEDivergenceKernel
    trial_variable = heat_flux
    variable = temperature
  []
  [h,h']
    type = MFEMTimeDerivativeVectorFEMassKernel
    variable = time_integrated_heat_flux
  []
  [-T,div.h']
    type = MFEMVectorFEDivergenceKernel
    trial_variable = temperature
    variable = time_integrated_heat_flux
    coefficient = -1.0
    transpose = true
  []
[]
```



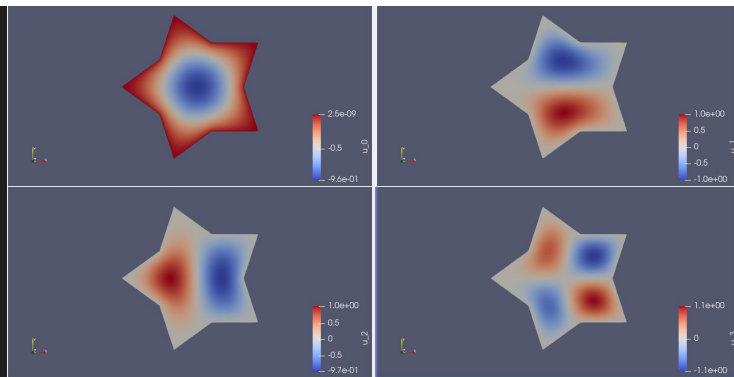
Eigenproblems and DG Systems

- Support for solving (real) eigenproblems has been added to MFEM-MOOSE
- DG problems can now also be set up in MOOSE, via the addition of boundary and interior face integrator contributions

```
[Problem]
  type = MFEMEigenproblem
  num_modes = 5
[]

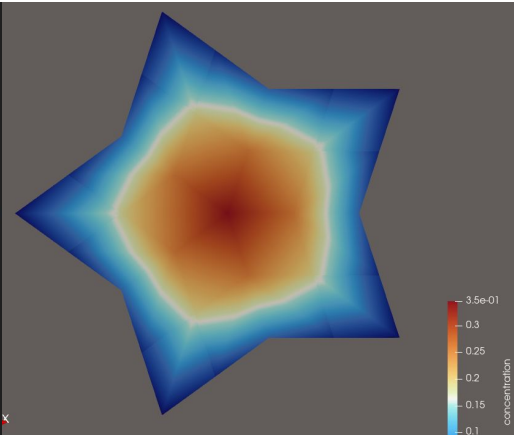
[Solvers]
  [boomeramg]
    type = MFEMHyprBoomerAMG
    print_level = 0
  []
  [LOBPCG]
    type = MFEMHyprLOBPCG
    preconditioner = boomeramg
    print_level = 0
    l_tol = 1e-10
    l_max_its = 300
    random_seed = 75
  []
[]

[VectorPostprocessors]
  [eigenvalues]
    type = MFEMEigenvaluesPostprocessor
  []
[]
```



```
[Kernels]
  [diff]
    type = MFEMDiffusionKernel
    variable = concentration
  []
  [forcing]
    type = MFEMDomainLFCKernel
    variable = concentration
  []
  [dg diff]
    type = MFEMDGDiffusionKernel
    variable = concentration
  []
[]

[BCs]
  [dg diff_bc]
    type = MFEMDGDiffusionBC
    variable = concentration
  []
[]
```



Custom Problem Composers & Operators

- New class for developers to easily wrap existing MFEM code into the MOOSE framework
- Can be used to define custom solvers, using MOOSE to construct as many (or as few) of the objects used as desired
- Facilitates MultiApp coupling of existing MFEM codes into MOOSE ecosystem
- See H. Rocha's talk (up next)

```
CustomProblemOperator::CustomProblemOperator(MFEMProblem & mfem_problem, mfem::Coefficient & coef)
: Moose::MFEM::ProblemOperator(mfem_problem), _coef(coef)
{
}

void
CustomProblemOperator::Init(mfem::BlockVector &)
{
    // Get the FE-space and Variable that were just built
    auto fes = _problem.getProblemData().fespaces.Get("H1");
    auto gridfunction = _problem.getGridFunction("u");

    // Boundary conditions
    fes->GetBoundaryTrueDofs(_boundary_dofs);

    // Build the linear form
    _b = new mfem::ParLinearForm(fes);
    _b->AddDomainIntegrator(new mfem::DomainLFIntegrator(_coef));
    _b->Assemble();

    // Build the bilinear form
    _a = new mfem::ParBilinearForm(fes);
    _a->AddDomainIntegrator(new mfem::DiffusionIntegrator);
    _a->Assemble();

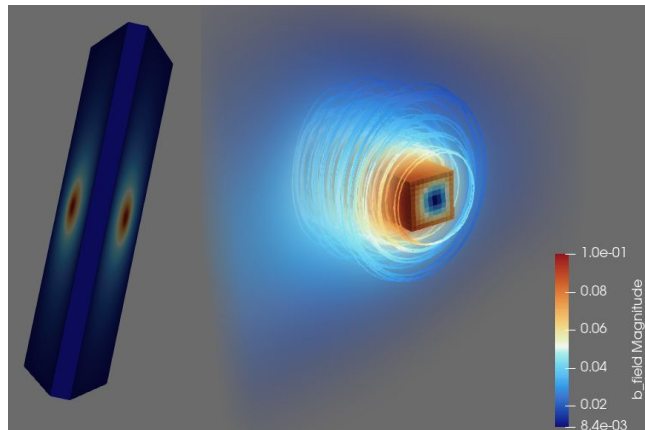
    // Form the linear system
    _a->FormLinearSystem(_boundary_dofs, *gridfunction, *_b, _problem_operator, _X, _B);

    void
    CustomProblemOperator::Solve()
    {
        // Set the operator and solve the equation
        _problem_data.jacobian_solver->SetOperator(*_problem_operator);
        _problem_data.jacobian_solver->GetSolver().Mult(_B, _X);

        // Set the data in the grid function
        auto grid_function = _problem.getGridFunction("u");
        grid_function->SetFromTrueDofs(_X);
    }
}
```

1. Introduction to MOOSE
2. MFEM in MOOSE
3. Recent Additions
4. Future Work
5. Summary

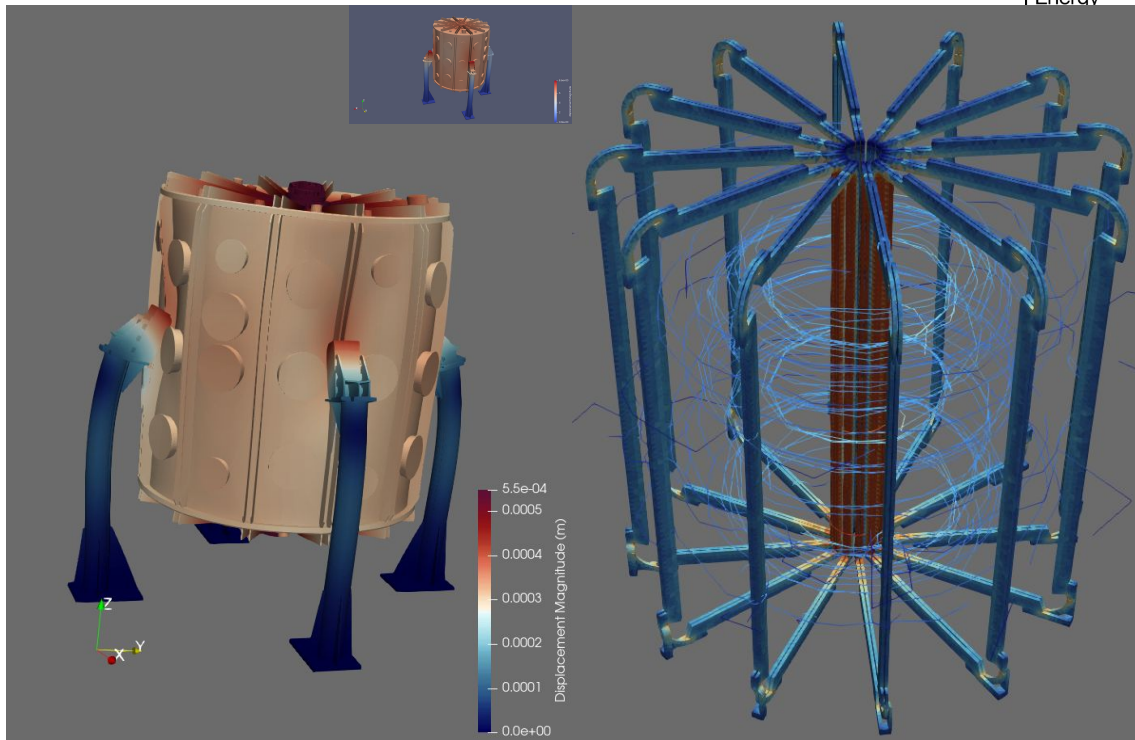
Upcoming



- Periodic mesh support (MOOSE PR #33337)
- Essential domain constraints (MOOSE PR #33624)
- Scalar integral constraints
- Perfectly matched layers (PML) and ‘infinite’ elements (MOOSE PR #33054)
- Generalized time integration schemes
- Block nonlinear problems and AD

- EM/mechanical simulations on full-tokamak meshes of MAST-U are planned for deployment and benchmarking on the new UK supercomputer **SUNRISE**:

- 1.4 MW
- 672 AMD MI355X GPUs
- 192 dual-socket Sapphire Rapids HBM 56-core CPU cluster



Simplified MAST vacuum vessel mesh deformed under gravitational loads consisting of 127M elements solved on CPU (bottom) and on GPU (8 NVIDIA A100s, top right) using MFEM in MOOSE. Rendered displacements increased by x1000. Mesh provided by William Ellis (UKAEA).

Magnetic flux density around simplified MAST-U TF coilset solved for using an A-formulation using MFEM in MOOSE.

1. Introduction to MOOSE
2. MFEM in MOOSE
3. Recent Additions
4. Future Work
5. Summary

Summary

- MFEM in MOOSE has seen heavy development over the last year, adding support for solving nonlinear systems, eigenproblems, DG problems, and AMR (and more!)
- Particular focus has been spent on improving interoperability between MFEM-based and libMesh-based problems in MOOSE, via new data transfer options and support for custom MFEM problem operators.
- Over the next year, we are targeting deployment of MFEM-MOOSE on the UK's new SUNRISE supercomputer on full-tokamak EM/mechanical simulations, with early access starting soon.

This work is part of the UKAEA's Advanced Computing Research Portfolio, funded by the Department of Energy Security and Net Zero in support of the UK's Fusion Strategy published in March 2026.